UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ 3303 – INTERNET PROGRAMMING

TOPIC 3 – JAKARTA SERVER PAGES (JSP) FORMERLY JAVASERVER PAGES

innovative • entrepreneurial • global



UTM JOHOR BAHRU

OBJECTIVES

Applied

- Code and test Jakarta Server Pages that use scriptlets, expressions, implicit request objects, page directives, JSP comments, and JSP declarations.

Knowledge

- Understand the concept, need and benefits of JSP.
- Explain how a JSP is executed in JSP lifecycle.
- Describe the use of JSP elements – scripting, directives, implicit object and action.
- Describe the way parameters are passed to a JSP when you use the Get method.
- Explain what is meant by a thread-safe JSP.
- Common JSP errors

UTM UNIVERSITI TEKNOLOGI MALAYSIA

INTRODUCTION

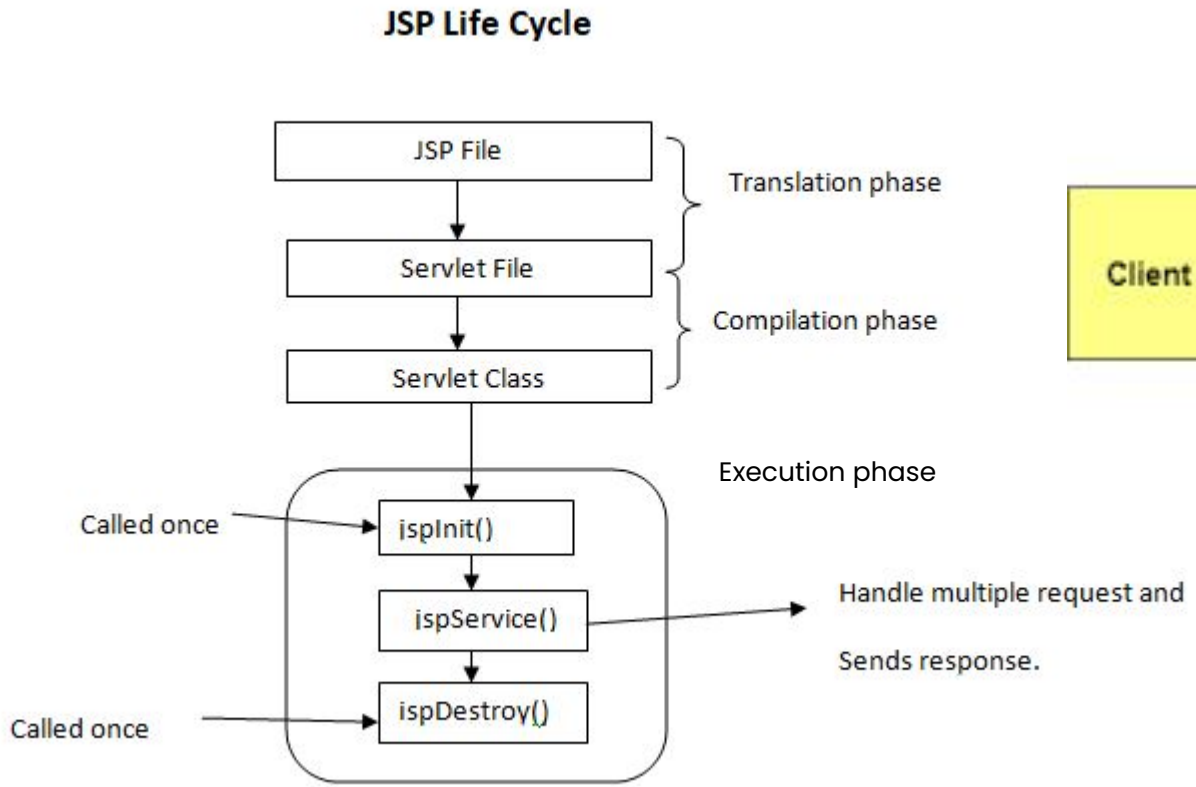
- Jakarta Server Pages (formerly JavaServer Pages) is a Java standard technology that developers use to write dynamic, data-driven web pages for Java web applications.
- JSP is built on top of the Java Servlet (*aka* Jakarta Servlet) specification and is one of the Java web technologies for server-side programming included for ongoing support and upgrades in Jakarta EE.
- From a coding perspective, the most obvious difference between JSP and servlets is that with servlets you write Java code and then embed client-side markup (like HTML) into that code. With JSP, you start with the client-side script or markup, then embed JSP tags to connect your page to the Java back end. This helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with <% and end with %>.

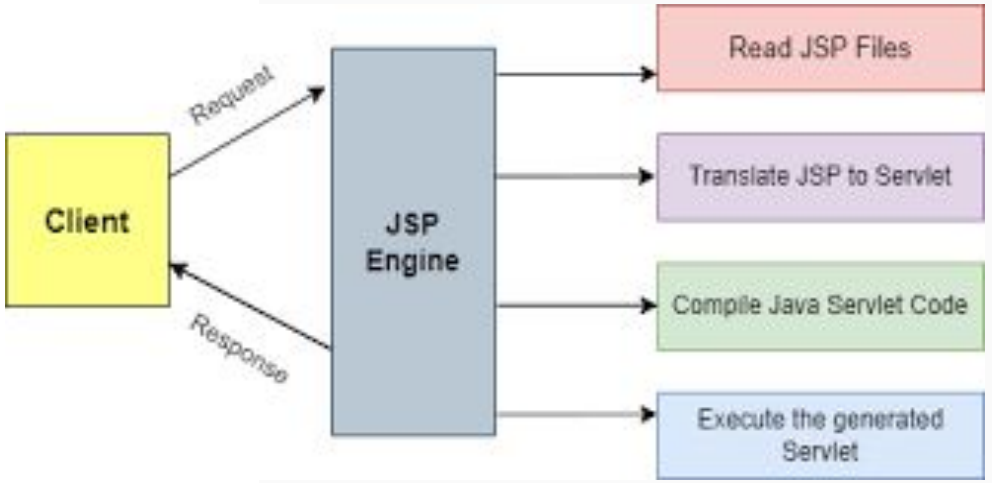
Reference: https://www.infoworld.com/article/3336161/what-is-jsp-introduction-to-javaserver-pages.html

UTM UNIVERSITI TEKNOLOGI MALAYSIA

TOPIC 3

JSP LIFECYCLE





Reference: https://www.codingninjas.com/codestudio/library/jsp-architecture-and-lifecycle

Reference: https://www.geeksforgeeks.org/life-cycle-of-jsp/

JSP LIFECYCLE

TOPIC 3

1. **Translation phase:** where the JSP file gets converted into the servlet file.
2. **Compilation phase:** where servlet file gets converted to servlet class.
3. **Execution phase:**
    - Class loading
    - Instantiation of the servlet object
    - Servlet container calls `jspInit()` method for initialization
    - Servlet container calls the `_jspService()` for processing the request
    - Clean up of the JSP, here servlet container will call `jspDestroy()` method.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

DO DEVELOPERS STILL USE JSP?

- Developers do still use JSP for some applications. It's a simpler technology than more modern approaches like Jamstack, or a template engine like Thymeleaf, but sometimes simple is the way to go.
- JSP is a core Java web technology. As developers, we can build JSP pages relatively quickly and easily, and they interact seamlessly with servlets in a servlet container like Tomcat.
- You will encounter JSP in older Java web applications. From time to time, you may still find it useful for building simple, dynamic Java web pages.
- As a Java developer, you should be familiar with JSP and how to use it in your programs.

Reference: https://www.infoworld.com/article/3336161/what-is-jsp-introduction-to-javaserver-pages.html

UTM UNIVERSITI TEKNOLOGI MALAYSIA

ADVANTAGE JSP OVER SERVLET

SERVLET

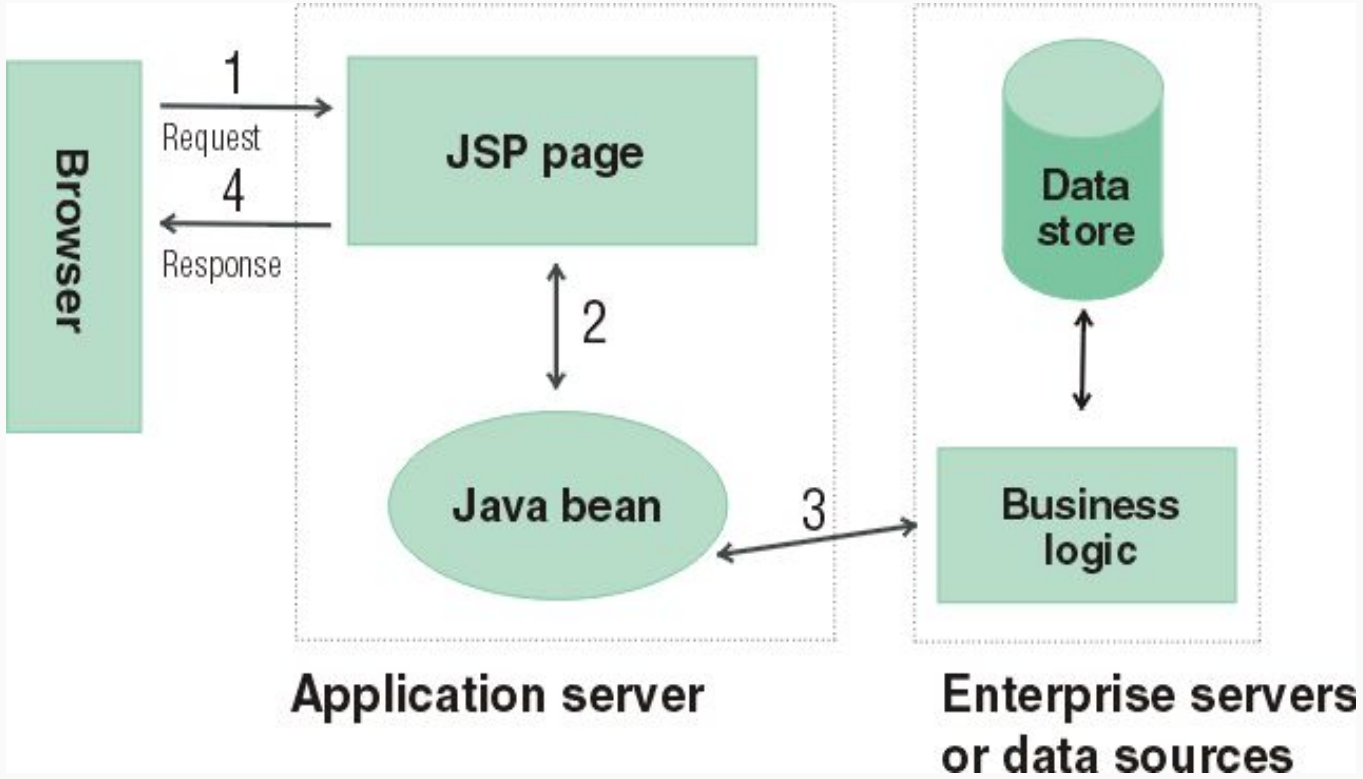
- Problem with servlet - content and presentation in one place
- A lot of out.println()
- HTML presentations are hard coded in java classes.
- Hard to redesign / change.

JSP

- Content and display logic are separated.
- Simplify web application development with JSP, JavaBeans and Action.
- Supports software reuse through the use of components.
- Automatic deployment – recompile automatically when changes are made to JSP pages.
- Platform independent.
- Easier to author web pages.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

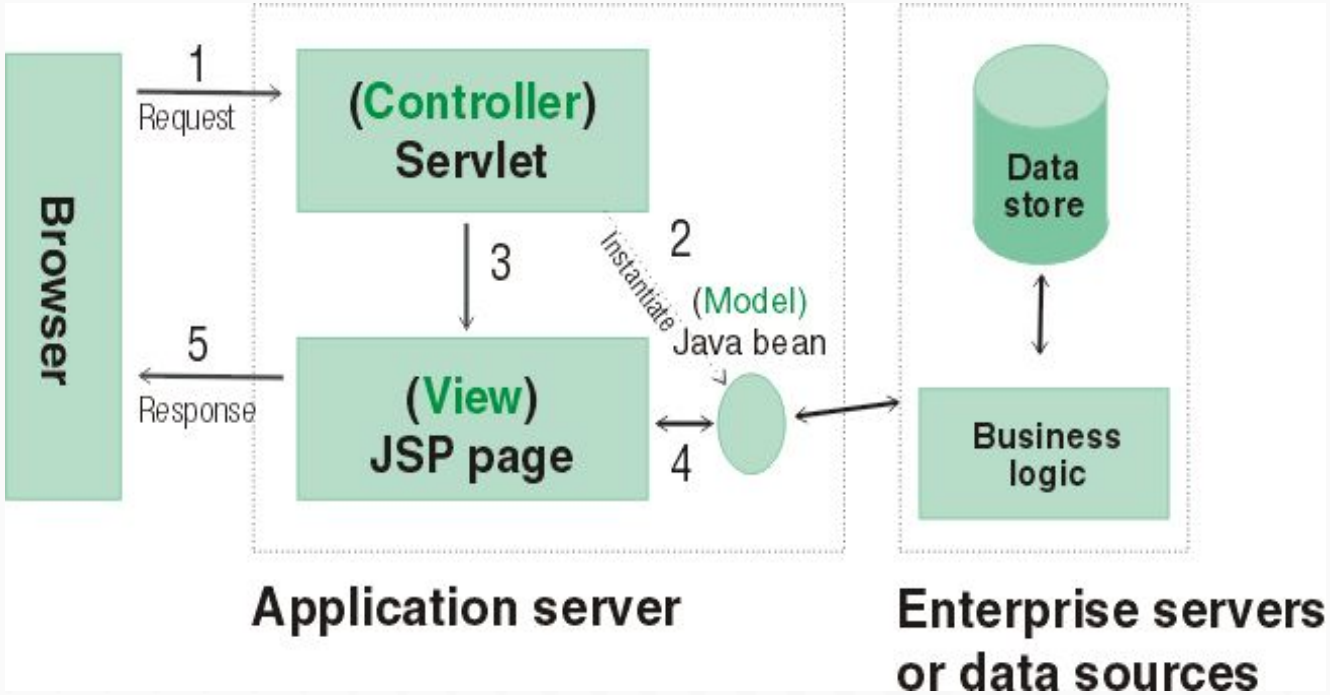
JSP MODEL 1 ARCHITECTURE



1. The browser sends a request to a JSP page.
2. The JSP page communicates with a Java bean.
3. The Java bean is connected to a database.
4. The JSP page responds to the browser.

Reference: http://www.setgetweb.com/p/rational/com.ibm.etools.struts.doc_6.0.0/topics/cstrdoc001.html

JSP MODEL 2 ARCHITECTURE



1. The browser sends a request to a servlet.
2. The servlet instantiates a Java bean that is connected to a database.
3. The servlet communicates with a JSP page.
4. The JSP page communicates with the Java bean.
5. The JSP page responds to the browser.

Reference: http://www.setgetweb.com/p/rational/com.ibm.etools.struts.doc_6.0.0/topics/cstrdoc001.html

WRITING JSP PAGES

- A simple JSP page consists of HTML markup embedded with JSP tags.
- JSP files have the .jsp extension.
- The JSP server (also called a JSP container) is configured to direct an HTTP request to a specific JSP page, which is then responsible for rendering an HTTP response.
- When the request arrives, the file is processed on the server and the HTML is rendered as the application view, a web page.
- The embedded JSP tags will be used to call server-side code and data.
- The ultimate product of the JSP is HTML that the client browser can understand as-is.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

JSP ELEMENTS

SCRIPTING ELEMENTS

- SCRIPTLET TAG
- EXPRESSION TAG
- DECLARATION TAG
- COMMENT TAG

DIRECTIVES ELEMENTS

- PAGE
- INCLUDE
- TAGLIB

IMPLICIT OBJECTS

- OUT
- REQUEST
- RESPONSE
- CONFIG
- APPLICATION
- SESSION
- PAGECONTEXT
- PAGE
- EXCEPTION

ACTION ELEMENTS

- JSP: FORWARD
- JSP: INCLUDE
- JSP: USEBEAN
- SET & GET PROPERTY

UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

1- Scriptlet Tag

- A scriptlet is a block of Java code executed during the request-processing time.
- In JSP, Java code can be written inside the JSP page using Scriptlet Tag.

Syntax: `<% java source code %>`

Example:

```
<html>
    <body>
        <% out.println ("Hello World"); %>
    </body>
</html>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

2- Expression Tag

- An expression is a shorthand notation that sends the evaluated Java expression back to the client (in the form of a String).
- Code placed within expression tag is written to the output stream of the response. So, no need to write out.println() to write data.

```
Syntax: <%= statement %>
Example:  <html>
          <body>
             <%= "Hello World" %>
          </body>
       </html>
Note: Do not end statement with semicolon (;)
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

A scriptlet and expression that display the value of the firstName parameter

Example:

```
<% String firstName = request.getParameter("firstName"); %>
The first name is <%= firstName %>
```

An expression that displays the value of the firstName parameter

Example:

```
The first name is <%= request.getParameter("firstName") %>
```

Notes:

- Within a scriptlet, you can code one or more Java statements. You must end each Java statement with a semicolon.
- Within a JSP expression, you can code any Java expression that evaluates to a Java object or to a primitive type. Since an expression isn't a statement, you don't end it with a semicolon.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

3- Declaration Tag

Used to declare variable and methods. The code written inside this tag is place outside the service() method of auto generated servlet. So, it does not get memory at each request.

Syntax: `<%! statement %>`

Example:

```
<html>
    <body>
        <%! int num = 3; %>
        <%= "Value is :" + num %>
    </body>
</html>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

TOPIC 3

3- Declaration Tag (continue)

- You can use *declaration tags* to declare instance variables and methods.
- Since instance variables aren't *thread-safe*, two threads may conflict when they try to read, modify, and update the same instance variable at the same time.
- In general, you should avoid coding instance variables for JSPs and servlets. Instead, you should use other thread-safe techniques for working with global variables.
- In general, you should avoid coding methods within JSPs. Instead, you should use some combination of JSPs, servlets, and regular Java classes.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

SCRIPTING ELEMENTS

4- Comment Tag

There are two types of JSP comment types.

- Hidden comments (JSP comments)
  - Visible only inside JSP page but not in generated HTML page.
  - **Syntax:** `<%-- This is a hidden JSP comment --%>`

- Comments within HTML content
  - Comments to be generated inside the output HTML page.
  - **Syntax:** `<!-- Comment inside the generated HTML -->`

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Java comments in a JSP scriptlet.

```
<% // get parameters from the request
String firstName = request.getParameter("firstName");
String lastName = request.getParameter("lastName");
String emailAddress = request.getParameter("emailAddress");
   /*
   User user = new User(firstName, lastName, emailAddress);
   UserIO.add(user, path);
   */
%>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

JSP Comment

```
<%-- comment text --%>
```

- A JSP Comment is removed by the JSP Engine at the translation time itself and the compilation unit doesn't contain them.
- The comment processing in JSP is very similar to what we have in any other language.
- Comments don't need to be compiled as we all know that they are not executed and they simply serve the purpose of making the source code more readable, and maintainable.

HTML Comment

```
<!-- comment text -->
```

- A JSP page considers this as a normal HTML tag and hence it doesn't remove them.
- All HTML comments are maintained in the response and one can easily see them just by viewing the page source of the rendered HTML page in the browser.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

When should I use JSP-style comments instead of HTML-style comments?

- Always use JSP-style comments unless you specifically want the comments to appear in the HTML that results from serving a JSP page.
- JSP Comments are removed by the JSP Engine during the translation phase (JSP Comments are not even part of the compilation unit) whereas the HTML Comments are treated like any other HTML tage and hence they are maintained throughout.
- This is reason why we can easily see the HTML comments in the browse by viewing the source of the page.

What about // or /*...*/ ? Are they JSP Comments ?

- No... they are only Java comments. Since, we are allowed to embed Java code into a JSP page, hence we can have them in a valid JSP page.
- These comments are treated the same way they are treated in a normal Java program.
- They all are removed before compilation and like JSP Comments they are also not part of the compilation unit.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

DIRECTIVE ELEMENTS

1- Page Directive

- A directive is used as a message mechanism to pass information from the JSP code to the container.
- Define attributes that apply to an entire JSP page.

```
Syntax: <%@ page attribute = "value" %>
```

```
Attributes: import, contentType, extend, info, buffer,
language, autoFlush, pageEncoding, isErrorPage,
errorPage
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

DIRECTIVE ELEMENTS

1- Page Directive (continue)

Example main attributes:

```
<%@ page language=java session=true %>
<%@ page import=java.awt.*,java.util.* %>
<%@ page isThreadSafe=false %>
<%@ page errorPage="errorPage.jsp" %>
<%@ page isErrorPage="true" %>
```

- To define the conditions that the JSP engine should follow when converting a JSP into a servlet, you can use a *JSP directive*.
- To import classes in a JSP, you use the import attribute of the *page directive*. This makes the imported classes available to the entire page.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

DIRECTIVE ELEMENTS

2- Include Directive

- Includes the contents of any resource (may be jsp file, html file or text file).
- It includes the original content of the included resources at page translation time.
- Reusability is the advantage.

```
Syntax: <%@ include file = "filename" %>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

DIRECTIVE ELEMENTS

3- Taglib Directive

- Used to define a tag library that defines many tags.
- Used the TLD (Tag Library Descriptor) file to define the tags.

```
Syntax: <%@ taglib uri = "urioftaglib"
        prefix = "prefixoftaglib" %>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

IMPLICIT OBJECTS

TOPIC 3

Variable	Type
out	JspWriter
request	HTTPServletRequest
response	HTTPServletResponse
config	ServletConfig
application	ServletContext
session	HTTPSession
pageContext	PageContext
page	Object
exception	Throwable

UTM UNIVERSITI TEKNOLOGI MALAYSIA

METHODS FROM THE REQUEST OBJECT

Method	Description
`getParameter(String param)`	Returns the value of the specified parameter as a string if it exists or null if it doesn't.
`getParameterValues(String param)`	Returns an array of String objects containing all of the values that the given request parameter has or null if the parameter doesn't have any values.
`getParameterNames()`	Returns an Enumeration object that contains the names of all the parameters contained in the request. If the request has no parameters, the method returns an empty Enumeration object.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

METHODS FROM THE REQUEST OBJECT

A scriptlet that determines if a checkbox is checked.

Example:

```
<% // returns the value or "on" if checked, null otherwise.
String rockCheckBox = request.getParameter("Rock");
if (rockCheckBox != null)
{
%>
   You checked Rock music!
<%
}
%>
```

METHODS FROM THE REQUEST OBJECT

A scriptlet that reads and displays multiple values from a list box.

Example:

```
<%
// returns the values of items selected in a list box.
String[] selectedCountries =
request.getParameterValues("country");

for (int i = 0; i < selectedCountries.length; i++)
{
%>
<%= selectedCountries[i] %> <br>
<%
}
%>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

METHODS FROM THE REQUEST OBJECT

A scriptlet that reads and displays all request parameters and values.

Example:

```
<%
Enumeration parameterNames = request.getParameterNames();
while (parameterNames.hasMoreElements())
{
String parameterName = (String)parameterNames.nextElement();
String parameterValue = request.getParameter(parameterName);
%>
<%= parameterName %> has value
<%= parameterValue %> <br>
<%
}
%>
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

ACTION ELEMENTS

TOPIC 3

Standard action are tags that affect the runtime behavior of the JSP and the response sent back to the client.

Look like single-tag HTML. The purpose is to make the JSP source look more like a standard HTML compared to a JSP source which is full with scriplets (raw java code).

UTM UNIVERSITI TEKNOLOGI MALAYSIA

ACTION ELEMENTS

Example:

```
<jsp:include page="URL" />
```

For including STATIC or DYNAMIC resources at request time.

```
<jsp:forward page="URL" />

<jsp:useBean id="name"
class="fully_qualified_pathname"
scope="{page|request|session|application}" />

<jsp:setProperty name="name" property="value" />

<jsp:getProperty name="name" property="value" />
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

When should I use a JSP <%@include@%> directive, and when should I use a <jsp:include> action?

- A JSP <%@include@%> directive (for example, <%@include file="myfile.jsp" @%>) includes literal text "as is" in the JSP page and is not intended for use with content that changes at runtime. The include occurs only when the servlet implementing the JSP page is being built and compiled.

- The <jsp:include> action allows you to include either static or dynamic content in the JSP page.
  - Static pages are included just as if the <%@include@%> directive had been used.
  - Dynamic included files, though, act on the given request and return results that are included in the JSP page. The include occurs each time the JSP page is served.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

COMMON JSP ERRORS

- HTTP Status 404 – File Not Found Error
- HTTP Status 500 – Internal Server Error

TIPS FOR FIXING JSP ERRORS

- Make sure the Tomcat server is running.
- Make sure that the URL is valid and that it points to the right location for the requested page.
- Make sure all of the HTML, JSP, and Java class files are in the correct locations.
- Read the error page carefully to get all available information about the error.

Credit:
Some of the content in this slide is based on text book- Murach's Java Servlets/JSP (3rd Ed.) © 2014, Mike Murach & Associates, Inc

UTM UNIVERSITI TEKNOLOGI MALAYSIA